Pokemon Data analysis

Problem Description :

Pokemon, a beloved and iconic franchise, has captured the hearts of millions of fans around the world since its inception. Created by Satoshi Tajiri and Ken Sugimori, Pokemon is a Japanese media franchise managed by The Pokemon Company, a collaboration between Nintendo, Game Freak, and Creatures Inc. The franchise centers around fantastical creatures called Pokemon, which humans known as Pokemon Trainers catch and train to battle each other for sport. Each Pokemon has its own unique abilities, types, and characteristics, making them diverse and captivating.

Dataset Information: The dataset 'data.csv' contains a wealth of information about various Pokemon species. It encompasses essential features, including:

1. **'Number':** The Pokemon's unique identification number.
2. **'Name'**: The name of the Pokemon.
3. **'Type_1':** The primary type of the Pokemon (e.g., Fire, Water, Grass, etc.).
4. **'Type_2':** The secondary type of the Pokemon (optional).
5. **'HP', 'Attack', 'Defense', 'Sp_Atk', 'Sp_Def', 'Speed':** The base stats of the Pokemon, which determine their performance in battles.
6. **'Generation':** The generation in which the Pokemon was introduced.
7. **'Legendary':** A binary variable indicating whether the Pokemon is legendary (1) or not (0).
8. **'hasMegaEvolution':** A binary variable indicating whether the Pokemon has a Mega Evolution (1) or not (0).
9. **'Color':** The color of the Pokemon, adding a vibrant touch to their appearance.
10. **'Egg_Group_1':** The primary egg group of the Pokemon, denoting compatibility for breeding.

11. **'Egg_Group_2':** The secondary egg group of the Pokemon (optional).
12. **'Body_Style':** The body style of the Pokemon, describing their unique physical traits and forms.
13. **'Height_m':** The height of the Pokemon in meters.
14. **'Weight_kg':** The weight of the Pokemon in kilograms.
15. **'Catch_Rate':** The catch rate of the Pokemon, indicating its rarity and ease of capture.
16. **'Pr_Male':** The probability of encountering a male Pokemon (NaN for genderless Pokemon).

Problem Statement: The objective of this data analysis project is to explore and analyze the Pokemon dataset to gain insights into various aspects of Pokemon characteristics. By leveraging the power of Python and popular data analysis libraries, we will delve into tasks such as data visualization, missing value analysis, correlation analysis, feature engineering, and building machine learning models to predict whether a Pokemon is legendary or not.

Motivation: The Pokemon franchise has a massive fan following, and its diverse array of Pokemon characters sparks curiosity and excitement among enthusiasts. This data analysis project aims to satiate the curiosity of Pokemon fans and provide fascinating insights into the world of Pokemon. By analyzing the dataset, we can uncover interesting patterns, correlations, and characteristics that make each Pokemon unique. Additionally, building predictive models will allow us to predict whether a Pokemon is legendary or not based on its attributes, adding an exciting dimension to the analysis.

Expected Outcomes: Through this analysis, we hope to achieve the following outcomes:

- **Data Visualization:** Present captivating visualizations of various Pokemon attributes, such as their types, colors, body styles, and more.
- **Insights into Legendary Pokemon:** Uncover patterns and traits that distinguish legendary Pokemon from non-legendary ones.
- **Feature Importance:** Identify the most influential attributes in determining whether a Pokemon is legendary.
- **Model Performance:** Evaluate the performance of different machine learning models in predicting legendary Pokemon with accuracy and AUC scores.

- **Data-Driven Fun:** Present intriguing findings that appeal to both Pokemon fans and data enthusiasts alike.

Let the data adventure begin as we embark on a journey to explore the enchanting world of Pokemon and discover the magic within the dataset! ⍰⍰

Possible Framework :

Introduction:

- Briefly introduce the Pokemon franchise and its significance.
- Explain the purpose of the data analysis project and the dataset used ('data.csv').

Data Loading and Exploration:

- Import the necessary libraries.
- Load the dataset ('data.csv') into a pandas DataFrame.
- Display the first few rows of the dataset to get an overview.
- Check the shape and info of the dataset to understand its size and data types.
- Visualize missing values using the missingno library to identify any data gaps.

Data Cleaning:

- Remove irrelevant columns like 'Type_2' and 'Egg_Group_2' that are not required for analysis.

Deal with missing values:

- For 'Pr_Male' attribute, impute missing values with the median.
- For 'Publisher' attribute, fill missing values with the most frequent publisher.

Data Exploration and Visualization:

- Explore the distribution of Pokemon types ('Type_1') using bar plots.
- Visualize the distribution of egg groups ('Egg_Group_1') using another bar plot.
- Analyze the distribution of 'Pr_Male' using line plots before and after imputation.
- Categorize and explore the 'Body_Style' attribute using a bar plot.
- Visualize the distribution of Pokemon colors using a pie chart.
- Plot box plots for 'HP', 'Attack', and 'Defense' attributes to identify outliers.
- Visualize the distribution of Pokemon height and weight using KDE plots.

Comparing Legendary Pokemon and Catch Rate:

- Create a pie chart to display the ratio of legendary and non-legendary Pokemon.
- Find the Pokemon with the top two highest total stats and create a treemap visualization.

Data Preprocessing:

- Replace boolean values (True/False) with binary values (1/0) to prepare for modeling.
- Merge similar types and colors to simplify the analysis:
- Merge Water, Ice into 'Water_Ice'.
- Merge Grass, Bug into 'Grass_Bug'.
- Merge Ground, Rock into 'Ground_Rock'.
- Merge Psychic, Dark, Ghost, Fairy into 'Psychic_Dark_Ghost_Fairy'.
- Merge Electric, Steel into 'Electric_Steel'.

Feature Engineering and Data Preparation:

- One-hot encode the categorical attributes 'Type_1' and 'Color' using pandas get_dummies.
- Define the feature set ('X') and the target variable ('y') for machine learning models.
- Split the dataset into training and testing sets using train_test_split.
- Initialize a class named 'modeling' for modeling and evaluation.

Modeling and Evaluation:

- Define a dictionary containing various classifiers (Logistic Regression, Random Forest, LGBM, XGBoost, CatBoost).
- Implement 5-fold cross-validation for each classifier using the KFold class.
- Scale the features using StandardScaler to improve model performance.
- Train each classifier on the training data and evaluate using accuracy and AUC scores.
- Present the comparison results of accuracy and AUC scores using bar plots.

Conclusion:

- Summarize the findings and insights gained from the data analysis.
- Highlight the performance of different models in predicting legendary Pokemon.
- Conclude the project with the significance of the analysis and potential future extensions.

Final Remarks:

- End the project on a positive note, appreciating the uniqueness of the Pokemon world.
- Encourage exploration and data analysis in other fascinating domains.
- Express gratitude to the Pokemon community and data enthusiasts for joining the data adventure.

Code Explanation :

*If this section is empty, the explanation is provided in the .ipynb file itself.

1. **Importing Libraries:** The code starts by importing the necessary Python libraries for data analysis and visualization, such as pandas, numpy, matplotlib, seaborn, plotly, and more. These libraries provide powerful tools to work with data and create stunning visualizations.
2. **Loading and Exploring the Dataset:** The next step is to load the dataset ('data.csv') into a pandas DataFrame called 'pokemon'. This DataFrame acts like a table that holds all the data from the dataset. The 'head()' function is used to display the first few rows of the DataFrame, giving us a glimpse of what the data looks like.
3. **Data Cleaning:** In this section, the code removes two columns ('Type_2' and 'Egg_Group_2') from the DataFrame as they are not needed for the analysis. It then uses the 'missingno' library to visualize and identify any missing values in the dataset. Missing values can be gaps or blanks in the data that need to be handled.
4. **Data Visualization:** This part of the code focuses on visualizing different aspects of the Pokemon data. It uses various types of plots, like bar plots and pie charts, to showcase interesting information about Pokemon types, egg groups, color distribution, and more. The 'plotly' library is used to create interactive visualizations.
5. **Data Exploration and Analysis:** The code explores attributes like 'Pr_Male', 'Body_Style', and 'Height', and visualizes them using line plots and KDE (Kernel Density Estimation) plots. This helps us understand patterns and trends in the data, like the distribution of Pokemon genders and heights.
6. **Comparing Legendary Pokemon and Catch Rate:** The code creates pie charts to compare the ratios of legendary and non-legendary Pokemon. It also identifies the top two Pokemon with the highest total stats using a treemap visualization.
7. **Data Preprocessing:** In this section, the code performs data preprocessing tasks like converting boolean values (True/False) to binary values (1/0) to prepare the data for machine learning models. It also merges similar types and colors to simplify the analysis.

8. **Feature Engineering and Data Preparation:** Feature engineering involves transforming the data into a format suitable for machine learning models. The code uses one-hot encoding to convert categorical attributes like 'Type_1' and 'Color' into numerical format. The 'X' DataFrame is created with all features, and the 'y' Series is assigned to the target variable.
9. **Modeling and Evaluation:** This is the most exciting part! The code introduces several machine learning models, such as Logistic Regression, Random Forest, LGBM, XGBoost, and CatBoost. It implements 5-fold cross-validation, which means dividing the data into five parts and using different parts for training and testing in each iteration. Each model is trained on the training data and evaluated using accuracy and AUC scores.
10. **Model Comparison:** The code presents a comparison of accuracy and AUC scores for each model using bar plots. These scores show how well each model performs in predicting legendary Pokemon. This comparison helps identify which model is the most effective for this analysis.
11. **Conclusion:** The code concludes the analysis by summarizing the findings and insights gained from the data exploration and machine learning models. It emphasizes the significance of the analysis and encourages further exploration in other exciting domains.
12. **Final Remarks:** The project ends with a positive note, appreciating the uniqueness of the Pokemon world and the fun of data exploration. It thanks the Pokemon community and data enthusiasts for joining this thrilling data adventure.

Future Work :

Step 1: Data Augmentation and Feature Engineering Data augmentation involves generating additional data points by applying various transformations to the existing data. For the Pokemon data analysis, we can perform data augmentation on attributes like 'Height', 'Weight', and 'Stats' to create more diverse and representative samples. Feature engineering can include creating new attributes like 'BMI' (Body Mass Index) from 'Weight' and 'Height' or deriving additional features from the existing ones. This step will enrich the dataset and improve the performance of machine learning models.

Step 2: Advanced Data Visualization In the current analysis, we have used various basic visualizations like bar plots, pie charts, and line plots. To enhance the visual storytelling, we can explore more advanced visualization techniques like 3D plots, sunburst charts, and choropleth maps to represent the data from different perspectives. This will provide a more engaging and interactive experience for users.

Step 3: Advanced Machine Learning Techniques While the project has already used several popular machine learning models, there are more advanced techniques to explore. For instance, we can implement deep learning models, such as neural networks, to predict legendary Pokemon with higher accuracy. Additionally, ensemble learning techniques, like stacking and blending, can be used to combine the predictions of multiple models for improved performance.

Step 4: Hyperparameter Tuning Hyperparameter tuning involves finding the optimal set of hyperparameters for machine learning models. By using techniques like Grid Search or Random Search, we can systematically search for the best combination of hyperparameters to maximize model performance. This step will fine-tune the models and make them more effective.

Step 5: Interpretability and Explainability To make the models more trustworthy and understandable, we can incorporate techniques for model interpretability and explainability. Methods like SHAP (SHapley Additive exPlanations) values or LIME (Local Interpretable Model-agnostic Explanations) can help explain the model's predictions and highlight the important features that influence the outcomes.

Step 6: Deployment and Web Application To share our analysis with a broader audience, we can deploy the project as a web application. Using frameworks like Flask or

Django, we can create an interactive and user-friendly interface where users can explore the Pokemon data, make predictions, and visualize the results. This will make the project accessible to Pokemon fans and data enthusiasts worldwide.

Step-by-Step Guide to Implement Future Work:

1. **Data Augmentation and Feature Engineering:**
- Generate new data points by transforming existing attributes like 'Height' and 'Weight'.
- Create additional features like 'BMI' based on 'Height' and 'Weight'.
2. **Advanced Data Visualization:**
- Explore 3D plots, sunburst charts, and choropleth maps to represent data in unique ways.
- Use Plotly Dash or Bokeh to create interactive visualizations.
3. **Advanced Machine Learning Techniques:**
- Implement deep learning models like neural networks for prediction.
- Explore ensemble learning techniques like stacking and blending.
4. **Hyperparameter Tuning:**
- Use Grid Search or Random Search to find optimal hyperparameters for models.
- Evaluate model performance with different hyperparameter combinations.
5. **Interpretability and Explainability:**
- Apply SHAP or LIME techniques to explain model predictions.
- Visualize feature importances and contributions to predictions.
6. **Deployment and Web Application:**
- Use Flask or Django to create a web application.
- Host the application on a web server or a cloud platform.
- Share the interactive analysis with the world!

Concept Explanation :

Concept Explanation: Machine Learning Models

Hello there, fellow Pokemon Trainer! Let's dive into the magical world of Machine Learning, where we'll uncover the secrets of the algorithms used in our Pokemon Data Analysis project! Are you ready for an adventure? Let's go!

1. Logistic Regression - The "Binary Classifier" Pokemon: Imagine you have a magic Pokeball that can predict whether a Pokemon is Legendary or not, just by looking at its attributes. That's what Logistic Regression does! It's like having a Pokemon that can sort others into two groups - Legendary or Not Legendary. It's like dividing the Pokemon world into two clans, and this Pokemon makes the decision!

2. Random Forest - The "Forest of Decisions" Pokemon: Picture a lush forest filled with many wise Pokemon. Random Forest is like a council of these Pokemon, each having a say in whether a Pokemon is Legendary. They vote, and the majority's decision is the final prediction. It's like a team of Pokemon using their combined wisdom to make the right call!

3. LGBM Classifier - The "Lightning Fast" Pokemon: Meet LGBM Classifier, the fastest Pokemon in the ML world! It can make predictions in a flash, like a bolt of lightning. It's super efficient and can analyze vast amounts of data quickly. With LGBM, you'll get your results in no time - it's like having a speedy Pikachu on your side!

4. XGBoost - The "Extreme Booster" Pokemon: XGBoost is like a Pokemon that loves challenges and constantly improves itself through training. It learns from its mistakes and becomes stronger with each battle. It's like a Pokemon that never gives up and gets better and better over time!

5. CatBoost - The "Cat Whisperer" Pokemon: Have you ever seen a Pokemon that can communicate with cats? Well, that's CatBoost for you! It understands the hidden language of the data and can predict Legendary Pokemon with uncanny accuracy. It's like having a Pokemon that can talk to cats and unravel their secrets!

6. Support Vector Machine (SVM) - The "Decision Line" Pokemon: SVM is like a Pokemon with a magic ruler that draws a line between Legendary and Non-Legendary

Pokemon. It finds the best line that separates them, like a Pokemon professor drawing a line between two different species. It's the genius of the ML world!

These Pokemon-like algorithms are the champions of Machine Learning, helping us make predictions, uncover patterns, and understand the fascinating world of Pokemon. They work tirelessly to assist us in our quest for knowledge, and with their help, we can become true Pokemon Masters! So, let's thank these ML Pokemon for their hard work and embark on more adventures in the world of data! Happy exploring, Trainer! ⍰⍰⍰

Exercise Questions :

1. What is the purpose of using SimpleImputer in the Pokemon Data Analysis project? Explain with an example.

Answer: The SimpleImputer is used to fill missing values in the dataset with a specified strategy. In our project, we used it to fill missing values in the "Pr_Male" column with the median value. For instance, if there are missing values in the "Pr_Male" column, the SimpleImputer will replace those missing values with the median value of the existing data in that column.

2. Why did we use One-Hot Encoding for the "Type_1" and "Color" columns? Describe the process and its benefits.

Answer: One-Hot Encoding is used to convert categorical variables like "Type_1" and "Color" into numerical form, making it easier for machine learning models to process the data. Each unique category in these columns is converted into a binary vector, where 1 represents the presence of that category and 0 represents the absence. It helps prevent any ordinal relationship assumptions among categories and allows ML models to understand the distinct categories.

3. How did we handle the issue of multicollinearity in the Pokemon Data Analysis project?

Answer: Multicollinearity occurs when two or more independent variables are highly correlated, which can negatively impact the performance of ML models. In our project, we used correlation matrices to identify highly correlated features and removed one of the correlated features to address the multicollinearity issue.

4. Explain the process of using K-Fold Cross Validation in the project. Why is it important?

Answer: K-Fold Cross Validation is used to evaluate the performance of machine learning models. It divides the dataset into K subsets or "folds," uses K-1 folds for training, and the remaining fold for validation. The process is repeated K times, ensuring that each fold is used for validation once. This helps in obtaining a more reliable estimate of the model's performance and reduces the risk of overfitting.

5. What is the purpose of StandardScaler in the project? How does it help ML models?

Answer: StandardScaler is used to standardize the features by removing the mean and scaling to unit variance. It ensures that all features are on the same scale, which is essential for many machine learning algorithms. StandardScaler helps prevent features with larger scales from dominating the model and ensures fair comparisons among features.

6. Describe the process of creating the radar plot for comparing "Arceus" and "Zekrom" stats.

Answer: The radar plot compares the stats of "Arceus" and "Zekrom" across different categories like HP, Attack, Defense, and Speed. For each category, the respective stat values are normalized to a scale of 0 to 1. The plot displays a polygon with vertices representing the categories and lines connecting the normalized values for each Pokemon. It allows us to visually compare the stats of "Arceus" and "Zekrom" across multiple attributes.

7. How does the CatBoostClassifier differ from other classifiers used in the project? Mention its unique features.

Answer: The CatBoostClassifier is unique because it can handle categorical features without the need for one-hot encoding. It is robust to overfitting and requires less parameter tuning compared to other classifiers. CatBoost also has a built-in logging feature that allows us to train the model silently, making it an efficient and user-friendly classifier.

8. Explain the process of data preprocessing for handling categorical data in the Pokemon Data Analysis project.

Answer: In data preprocessing, we used Label Encoding to convert categorical variables like "Platform" and "Genre" into numerical form. Label Encoding assigns a unique integer to each category in the column. Additionally, we used One-Hot Encoding for "Type_1" and "Color" columns, converting them into binary vectors. These steps ensure that categorical data is transformed into a format that ML models can understand.

9. How did we handle the issue of class imbalance in the Pokemon Data Analysis project?

Answer: Class imbalance occurs when the number of instances of different classes in the target variable is not balanced. In our project, we used K-Fold Cross Validation to train the models on different subsets of the data, ensuring that each fold has a balanced representation of the target classes. Additionally, we used evaluation metrics like Accuracy and AUC to measure the performance of the models on imbalanced classes.

10. What are the main differences between Random Forest and XGBoost? When should we choose one over the other?

Answer: Random Forest is an ensemble method that creates multiple decision trees and combines their outputs to make predictions. XGBoost, on the other hand, is a gradient boosting algorithm that builds multiple trees sequentially, correcting the errors of previous trees. Random Forest is suitable for tabular data and can handle missing values well, while XGBoost is preferred for high-dimensional datasets and offers better performance for structured data with many features.